ChatGPT

AWS Agent Core (Amazon Bedrock AgentCore)

Executive Summary: Amazon Bedrock AgentCore (often referred to as "AWS Agent Core") is a new managed service for deploying and operating AI agents in production [1] [2]. AgentCore provides a serverless **runtime** that hosts agent code, **memory** for context (short- and long-term), a **gateway** for tool integration, and full **observability** – all with enterprise-grade security and automatic scaling [1] [2]. Unlike traditional AWS agents (e.g. the SSM Agent on EC2 or the Greengrass agent on IoT devices), AgentCore is cloud-native: you author agent logic (e.g. in Python or containerized code), and AgentCore handles provisioning, versioning, and isolated sessions up to 8 hours [3] [4]. Agents can integrate with any APIs or Lambda functions via AgentCore Gateway, and can use Bedrock foundation models for decision-making. AgentCore supports secure identity (AWS IAM or OAuth2), VPC connectivity, and built-in logging/metrics (CloudWatch and X-Ray) [5] [6]. This report explains AgentCore's architecture, features, differences from SSM and Greengrass agents, and typical use cases. We then give a detailed CLI walkthrough to create, deploy, and invoke a simple AgentCore agent, including minimal IAM policies and troubleshooting tips.

What is AgentCore?

Amazon Bedrock AgentCore is **"an agentic platform for building, deploying, and operating effective agents securely at scale—no infrastructure management needed."** [1]. In practice, AgentCore lets developers package their AI agents (for example, chatbots or decision-making bots) and run them in a managed, serverless environment. Each agent runs as a containerized application (via a **runtime**) that maintains conversational context and calls out to tools. Key features include: - **Runtime hosting:** AgentCore spins up isolated micro-VMs for each user session (up to 8 hours) [3]. Sessions preserve state (via memory) and are fully sandboxed to prevent data leakage between users. - **Memory:** A built-in memory service supports both short-term (session) and long-term memory. Short-term memory keeps recent turns for a conversation; long-term memory extracts and stores facts (e.g. user preferences) across sessions [7]. This lets agents "remember" past interactions and personalize responses. - **Gateway (Tool Integration):** AgentCore Gateway makes APIs, Lambda functions, and services available as **MCP-compatible tools** for agents [8]. Agents can discover and invoke these tools (for example, querying a database or sending a message to Slack) with secure credential handling and semantic search. - **Identity & Security:** AgentCore supports AWS IAM (SigV4) or OAuth2 for inbound auth. Outbound calls (to tools/services) can use API keys or OAuth with managed token injection. Enterprise features like VPC networking and PrivateLink are supported [5]. All agent code runs with assigned IAM execution roles that grant only the needed AWS permissions. - **Observability:** By default, AgentCore emits OpenTelemetry traces and metrics to CloudWatch and X-Ray [6]. This provides detailed logs of agent actions, model invocations, and performance metrics. - **Versioning & Endpoints:** Each time you update your agent, AgentCore creates a new immutable version. A stable **DEFAULT** endpoint always points to the latest version, and you can create custom endpoints (e.g. "dev" vs "prod") to control rollouts [9]. - **Additional Features:** The platform supports asynchronous background tasks (up to 8h) and streaming responses. It also provides experimental features like Policy (natural-language policies enforced at runtime) and Evaluations (built-in agent quality metrics) [10] [11].

These components work together as shown below:

Figure: High-level AgentCore architecture (courtesy AWS). The AgentCore Runtime hosts your agent code, which communicates with Bedrock models. AgentCore also provides Memory for context, a Gateway to expose tools/APIs to the agent, an optional Code Interpreter (for safe code execution), and built-in Observability (CloudWatch/X-Ray) [12] [3]. End users authenticate (via Amazon Cognito or IAM) and call the agent through a REST or WebSocket endpoint. The service runs entirely in AWS (no servers to manage) and supports VPC networking for secure access to internal resources [5] [12].

Supported Platforms and Frameworks

AgentCore is **fully cloud-hosted**: there is no software to install on user machines. Agent code is packaged as either zipped Python code or a container image, and deployed to AWS (Amazon ECR/CodeBuild by default) [13] [14]. The AgentCore CLI and SDK support popular AI frameworks and languages. For example, the Python SDK and CLI can bootstrap agents written with **Strands Agents, LangGraph, CrewAI, LlamaIndex**, etc [15] [16]. Under the hood, AgentCore simply routes inputs to Bedrock (or OpenAI/Google) models, so any language that can be containerized will work. The CLI itself (`agentcore`) is a Python package (works on Linux, macOS, Windows) [17]. According to AWS, AgentCore is available in all major regions (e.g. us-east-1, eu-west-1, ap-south-1, etc.) [18] with full support for the core features (runtime, memory, gateway, identity, observability).

Architecture and Components

AgentCore's architecture centers on the **AgentCore Runtime** – a managed environment that hosts your agent (or tool) code [19]. Each runtime is versioned and tied to an **endpoint** (an API address). The DEFAULT endpoint always points to the latest version. Agents communicate via REST or WebSockets, receiving user inputs and returning responses.

- **Runtime & Sessions:** When you deploy an agent, AgentCore creates a runtime container. Each user interaction occurs within a **session**, identified by a `runtimeSessionId`. Each session runs in its own isolated micro-VM with dedicated CPU, memory, and filesystem (scrubbed at session end) [3].

This isolation ensures no cross-talk between users and meets enterprise security needs. Sessions last up to 8 hours; if you reuse the same session ID, the conversation context is preserved.

- **Memory Service:** Agents are stateless by default, so AgentCore provides a **Memory store** to give them stateful context [20]. Short-term memory automatically saves all turns within a session. Long-term memory extracts key facts (preferences, facts, summaries) across sessions and stores them in a vector store. This lets agents remember, for example, that a user prefers window seats or past conversation topics [7] [21]. (See *Figure:* agents with vs without memory below.)

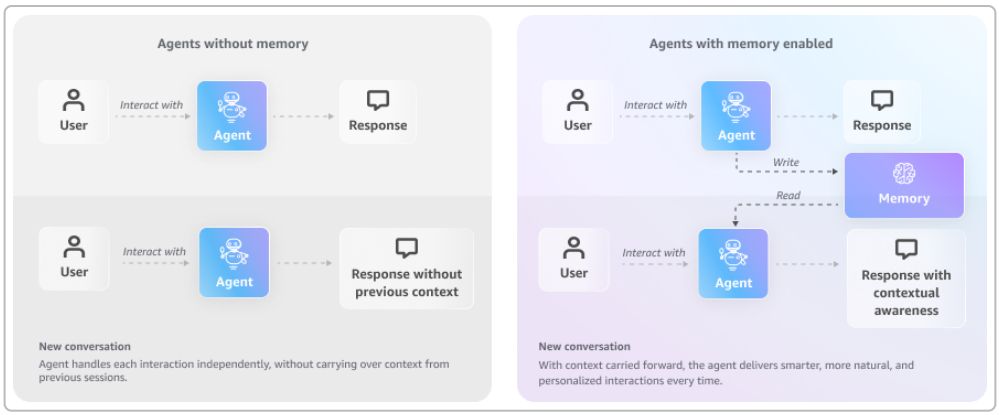

Figure: Agents without memory (left) lose all history each turn, while with memory (right) the agent writes to/reads from a memory store to maintain context and personalized facts across sessions [20].

- **Gateway (Tool Integration):** Real-world agents need external tools (databases, APIs, SaaS services). AgentCore Gateway lets you **publish any API or Lambda function as an agent tool** through a single secure endpoint [8]. Gateway supports OpenAPI, Smithy models, or direct Lambda handlers. It handles ingress auth (verifying the agent's identity), egress auth (OAuth token injection, API keys, etc.), and can semantically index tools so agents can "discover" the right tool. For example, an agent could call a Jira API or a custom document-search API via the gateway. Gateway scales automatically and logs all agent-to-tool calls.

- **Identity & Authentication:** AgentCore uses **AgentCore Identity** for secure agent calls. Inbound requests can be authenticated via AWS IAM (SigV4) or OAuth2 tokens from external IdPs (e.g. Amazon Cognito, Okta). You configure allowed audiences and clients when setting up OAuth. Outbound calls (when an agent calls a tool) are made either with user-delegated credentials or a service credential managed by AgentCore, keeping secrets out of your code. The gateway handles OAuth flows and token refresh seamlessly.

- **Policy Engine:** A new feature is the policy engine (Cedar-based) integrated with Gateway. You can write high-level policies (in natural language) to restrict what tools or actions an agent may use [10]. For example, you might forbid an agent from issuing refunds above $1,000 or limit it to read-only data. The policy engine checks every agent action in milliseconds to enforce boundaries.

- **Observability:** AgentCore automatically collects logs and metrics. Each agent invocation and agent-to-model call is traced via OpenTelemetry. CloudWatch metrics (such as agent latency, errors, cost

metrics) and AWS X-Ray traces are available by default [6]. You can also enable streaming responses and bidirectional WebSocket streaming for low-latency interaction.

Overall, **AgentCore is a purpose-built, managed stack** for AI agents, combining a runtime, tool integration, memory, and observability under AWS control. It removes the need for customers to provision servers or build scaffolding, freeing them to focus on agent logic.

Supported Platforms vs. Other AWS Agents

AgentCore is fundamentally different from other "agents" in AWS. In the AWS world:

- **AgentCore (Amazon Bedrock AgentCore):** Cloud-hosted, serverless. You deploy agent code (usually in Python or as a container) to AWS. Runs AI-agent workloads (conversational or goal-oriented) using LLMs and tools. Requires AWS credentials to invoke via API. Integrates with memory, gateway, and cloud models.

- **SSM Agent (AWS Systems Manager Agent):** A process that runs *on* EC2 instances or servers (Linux, Windows, macOS) [22] [23]. Its purpose is infrastructure management: it polls the Systems Manager service and executes commands (patching, scripts, inventory) on the host. It is pre-installed on many official AMIs. SSM Agent is not AI-aware – it's for OS management, not conversation.

- **IoT Greengrass (AWS IoT Core Greengrass Edge Agent):** Software that runs on IoT devices (usually Linux/ARM) to enable local compute. Greengrass Core software can host Lambda functions, sync data with AWS, and manage IoT communications. It is meant for edge scenarios, offline processing, sensor data, etc., not general AI chatbots.

The table below summarizes the differences:

Feature	AWS AgentCore	SSM Agent	AWS IoT Greengrass Agent
Purpose	Deploy and run AI agents (chatbots, assistants, workflows) using generative models and tools [1].	Manage instances/servers: run commands, scripts, patching, inventory.	Edge computing on IoT devices: run local Lambda, data sync, messaging.
Environment	Fully managed in AWS (serverless). Runs in AWS VPC or Public subnets; no software on customer servers.	Runs on EC2/on-prem instances (Linux, Windows, macOS) [22] [23].	Runs on Greengrass Core devices (embedded Linux).

Feature	AWS AgentCore	SSM Agent	AWS IoT Greengrass Agent
Deployment Model	Packaged as code or container and pushed to AWS (ECR/CodeBuild) via CLI/SDK [13] [24]. AWS sets up runtime, ECR, IAM roles.	Installed as an OS service. AWS AMIs often have it preinstalled; can be manually installed on other hosts [23].	Installed as a runtime (nucleus) on devices. Deployment via AWS console or CLI.
Invocation	Invoked via AgentCore endpoints (API Gateway/ WebSocket) with IAM or OAuth auth [25].	Controlled by Systems Manager (SSM) service. Jobs are pushed to agent via AWS APIs.	Connects to AWS IoT Core. Devices subscribed to MQTT topics and receive commands/updates.
Identity/ Security	Uses IAM roles for runtime execution (with limited Bedrock and logs permissions) [26] [4]. Supports OAuth2 (Cognito, etc.) for user access. VPC & PrivateLink support [5].	Uses instance IAM role (Node's EC2 profile) for SSM API calls. Communicates with SSM Message Gateway endpoints [22]. No built-in notion of sessions or memory.	Uses X.509 certificates for device auth (AWS IoT). Can use IAM role for Greengrass if linked with AWS resources.
Languages/ Frameworks	Agent code typically written in Python (3.10+) or other languages in a container [13]. Works with AI frameworks (Strands, LangGraph, CrewAI, etc.) [15] [16].	SSM plugins and scripts are OS-level (Bash, PowerShell, Python, etc.). The agent itself is closed-source.	Greengrass runs AWS Lambda functions (Python, Node.js, Java, C, etc.) on the device.
Logging/ Monitoring	Logs and metrics go to CloudWatch and X-Ray by default [6]. Agents are multi-tenant (per-session isolation).	Can publish logs/metrics via CloudWatch (if configured). SSM session logs optionally sent to S3 or CloudWatch.	Greengrass device logs can be sent to CloudWatch or local log files.

In summary, SSM and Greengrass agents are for infrastructure and IoT management respectively; they run on customer machines and are not AI agents. AgentCore, by contrast, is a cloud-native AI agent platform integrated with Bedrock models and specialized features like memory and tool gateways [1] [8].

Typical Use Cases

AgentCore is suited for any scenario where an AI agent needs to **carry on multi-turn conversations or orchestrate tasks securely**. Common examples include [21] [27]:

- **Conversational assistants:** Chatbots for customer support or internal helpdesks that remember user history. For example, a support agent that recalls previous orders or preferences [21].

- **Multi-step workflows:** Task-oriented agents that manage processes, e.g. processing an invoice or troubleshooting a system. Memory lets the agent track workflow state across steps [21].
- **Multi-agent orchestration:** Complex tasks managed by specialized agents. For instance, a multi-agent supply-chain optimization (inventory agent, demand-forecast agent, scheduling agent) all sharing memory [21].
- **Autonomous planning:** Agents acting on longer-term goals (e.g. an autonomous vehicle route planner that learns from past trips) [21].
- **Intelligent tools:** Agents that query company data, generate code, or perform analysis by calling APIs via the gateway. For example, an agent could query a CRM system for contact info or run SQL queries on a database through a Gateway-wrapped Lambda.

Companies are already using AgentCore for advanced scenarios. For example, device manufacturers use it to create maintenance assistants, finance firms build multi-agent analyzers, and security teams build incident-response bots [28] [29]. In all cases, AgentCore's integrated memory and policy engine help enforce business rules and context.

Security, IAM and Networking

Security is a core design goal. Each agent runs in an **isolated microVM** and only interacts with AWS APIs or tools per its IAM permissions. Key points:

- **Execution IAM Roles:** Every AgentCore runtime assumes an IAM role that you provide. This role grants the permissions needed to run the agent (invoke Bedrock models, read/write logs/metrics, pull container images, etc.) [26] [4]. For least privilege, restrict the role's actions and resources. For example, allow only CloudWatch Logs actions on the `arn:aws:logs:*:*:log-group:/aws/bedrock-agentcore/runtimes/*` log group [26], and only `bedrock:InvokeModel` on the specific Bedrock models used [4]. The official docs provide example policies showing these permissions [4] [26]. You must also update the role's **Trust Policy** to allow `bedrock-agentcore.amazonaws.com` to assume it [30].
- **Gateway and Tool Permissions:** Tools exposed via Gateway run under separate roles (with their own IAM policies). AgentCore can inject credentials or tokens when an agent calls a tool. You configure which AWS Identity or OAuth principal the agent uses for external API calls.
- **Networking (VPC):** By default, agents access the Internet to reach Bedrock and public APIs. For private access, AgentCore supports **VPC mode**. You can configure your agent runtime with `--vpc`, providing private subnets and security groups [31]. In VPC mode, the runtime is placed inside your private network (via AWS PrivateLink) so it can access RDS, on-prem resources, etc. AWS also supports VPC endpoints for Bedrock and related services.
- **Logging/Monitoring:** AgentCore logs all agent interactions. The CLI (`agentcore status` and `invoke`) even provides quick links to CloudWatch logs. Internally, AgentCore uses **AWS X-Ray** and CloudWatch Metrics to trace and record execution. The execution role must allow X-Ray (`xray:PutTraceSegments`) and CloudWatch (`PutMetricData`) [32]. Audit data (who invoked which agent, etc.) is available via CloudTrail.
- **Data Protection:** All data between users and AgentCore can be encrypted (HTTPS endpoints). Memory data is stored in encrypted AWS services (it uses managed vector stores or DynamoDB behind the scenes). Session isolation ensures that ephemeral data is wiped after use.
- **Scalability:** AgentCore auto-scales. You pay only when sessions are active. AWS isolates each session on a microVM and can run many in parallel. There's no fixed number of concurrent agents – capacity

grows with demand. Since each version is immutable, updates happen with zero downtime. (AWS advertises "dynamic scaling" for AgentCore [2].)

Security best practices for AgentCore parallel those for other AWS services: use least-privilege IAM roles, integrate with your IdP (e.g. enforce MFA via Cognito), and monitor with CloudWatch Alarms. Unlike SSM or Greengrass, there is no privileged agent on customer hosts – the entire system is in AWS's control, reducing the on-host attack surface.

Step-by-Step CLI Example

Below is a practical example of deploying a simple AgentCore agent that uses a Bedrock model. We assume you have an AWS account with Bedrock/AgentCore enabled and a suitable IAM user.

1. **Prerequisites:** Install the AWS CLI and configure your AWS credentials/region. Install Python 3.10+ and `docker` (Docker or Podman) on your local machine [13] [17].

2. **Install the AgentCore CLI:**

   ```
   pip install bedrock-agentcore-starter-toolkit
   ```

   (This provides the `agentcore` command.) If `pip` requires update, see AWS docs [17].

3. **Create agent scaffold:**
   Use `agentcore create` to bootstrap an agent. For example, with a simple StrandsAgent template:

   ```
   agentcore create --name my-agent --deployment-type direct_code_deploy --
   framework strands --model amazon.bedrock.ftt-refined:XYZ
   ```

   This command creates a folder `my-agent/` with example code (`agent.py`) and a `config.json`. You can customize the agent logic in `agent.py` (e.g. change prompts or tool calls).

4. **Configure IAM role (least privilege):**
   The starter toolkit will try to create needed IAM roles if you have permissions. Alternatively, pre-create a role with an inline policy like:

   ```
   {
     "Version": "2012-10-17",
     "Statement": [
       {"Effect":"Allow","Action":
   ["logs:CreateLogStream","logs:PutLogEvents"],"Resource":
   ["arn:aws:logs:*:*:log-group:/aws/bedrock-agentcore/runtimes/*"]},
       {"Effect":"Allow","Action":
   ```

```
["cloudwatch:PutMetricData"],"Resource":"*","Condition":{"StringEquals":
{"cloudwatch:namespace":"bedrock-agentcore"}}},
    {"Effect":"Allow","Action":["bedrock:InvokeModel"],"Resource":
["arn:aws:bedrock:*::foundation-model/*"]},
    {"Effect":"Allow","Action":["bedrock-
agentcore:GetWorkloadAccessToken"],"Resource":["*"]}
  ]
}
```

Attach this to a role (trusted by `bedrock-agentcore.amazonaws.com`). In practice, the toolkit's managed policy `AmazonBedrockAgentCoreFullAccess` can be used for testing. For least privilege, scope the above actions to specific resources [4] [26].

5. **Deploy the agent:**

```
cd my-agent
agentcore deploy
```

By default, this uses AWS CodeBuild to build an image and create the AgentCore runtime. You should see output like "□ Build succeeded" and then an ARN of your AgentCore endpoint. The CLI will also print CloudWatch log tail commands and a "GenAI Dashboard" link if OTEL is enabled.

6. **Check status:**

```
agentcore status --agent my-agent
```

This shows the deployment status. A ready endpoint indicates you can invoke the agent.

7. **Invoke the agent:**

```
agentcore invoke --agent my-agent '{"user_input": "Hello"}'
```

The CLI will call the agent, creating a new session ID, and display the response. It also shows request/session IDs and a CloudWatch tail command. Use the AWS CLI or console to view detailed logs if needed.

8. **Verify operation:**

9. Confirm the agent's output is as expected (e.g. echoes or answers the prompt).
10. In CloudWatch Logs, find the log group `/aws/bedrock-agentcore/runtimes/…` to inspect the execution.

11. Check CloudWatch Metrics (namespace `bedrock-agentcore`) for invocation metrics.

12. **Optional – Add Memory:** By default memory is enabled. If you disabled it (`--disable-memory`), you can enable with:

    ```
    agentcore memory create --agent my-agent
    ```

    Then redeploy. This provisions short-term memory (and optionally long-term) for the agent [33].

13. **Cleanup:**
    When done, destroy resources to avoid charges:

    ```
    agentcore destroy --agent my-agent --force
    ```

    This deletes the runtime, CodeBuild project, ECR images (if `--delete-ecr-repo` is given), and related resources.

Sample Workflow (Remote Command)

As an example, you could create an agent that executes AWS CLI commands via SSM. Define a tool in Gateway that wraps the AWS Systems Manager "SendCommand" API. The agent's prompt could say: "Use the *run_aws_cli* tool to execute system commands on a server." In this way, the agent calls your custom Lambda (via Gateway) which in turn uses boto3/SSM to run a command on an instance. Similarly, telemetry collection or software deploys can be invoked via agent tools. (Designing these tools involves writing AWS Lambda or API wrappers and adding them to AgentCore Gateway.)

Troubleshooting and Common Issues

- **IAM errors:** The most common issues are permission-related. If you see *"Access Denied"* for a CloudWatch or Bedrock call, check that the execution role has the needed actions (e.g. `logs:PutLogEvents`, `bedrock:InvokeModel`). Use CloudWatch Logs to view agent runtime errors. Ensure the role's trust policy allows `bedrock-agentcore.amazonaws.com` [30].

- **Deployment failures:** If `agentcore deploy` fails in CodeBuild, examine the CodeBuild logs. Common problems include missing permissions (e.g. ECR push) or network/VPC issues. You may need to grant `ecr:*` permissions or allow CodeBuild the right VPC/subnet if using VPC mode.

- **Invocation timeouts:** Long-running agents (or heavy LLM calls) may exceed default timeouts. Ensure your agent handles context quickly or enable streaming responses. For extremely long tasks, use the `/ping` endpoint to extend session life.

- **Networking issues:** In VPC mode, ensure the subnets have NAT/Internet access (for Bedrock calls) or the necessary VPC endpoints. Misconfigured security groups can block agent access to Bedrock or Gateway.

- **Memory provisioning:** When enabling memory, provisioning can take a couple of minutes. The CLI waits for memory to become ACTIVE. If memory creation fails, check IAM (DynamoDB, Elasticsearch or vector DB permissions) and service quotas.

- **CLI problems:** If the `agentcore` CLI isn't recognized, ensure Python 3.10+ is used. Some users must upgrade `pip` or add `~/.local/bin` to PATH. The CLI's `--verbose` flag can help debug issues.

- **Quota limits:** AgentCore quotas (sessions, endpoints, etc.) are similar to other AWS services. By default, you get a limited number of runtimes per region. Contact AWS support to raise limits for large deployments.

Architecture Diagram (Mermaid)

Below is a simple architecture diagram of AgentCore:

```
flowchart LR
    subgraph Client
      U(User)
      U -->|OAuth/IAM| Frontend(App)
    end
    subgraph AWS
      Cognito((Amazon Cognito))
      API(API Gateway)
      LambdaAuth(Lambda)
      AgentCoreRuntime((AgentCore Runtime))
      Memory[(AgentCore Memory)]
      Gateway((AgentCore Gateway))
      CodeInterp([Code Interpreter])
      CloudWatch[CloudWatch Logs/Metric]
      Tools[(External Tools/API)]
    end
    Frontend --> Cognito
    Cognito --> API
    API -->|InvokeAgentRuntime| AgentCoreRuntime
    AgentCoreRuntime --> CodeInterp
    AgentCoreRuntime --> Memory
    AgentCoreRuntime --> Gateway
    Gateway --> Tools
    AgentCoreRuntime --> CloudWatch
```

Figure: Simplified AgentCore architecture. Users (via a front end or API client) authenticate (Cognito/IAM) and invoke the AgentCore Runtime endpoint. The runtime interacts with Bedrock foundation models, uses the Memory store for context, and calls external Tools through the AgentCore Gateway. Logs and metrics flow to CloudWatch. (This diagram omits multi-agent and identity components for brevity.)

Comparison Table: AgentCore vs Other AWS Agents

Feature/Aspect	**AgentCore (Bedrock)**	**SSM Agent (Systems Manager)**	**Greengrass Core (IoT Edge)**
Primary Use Case	AI conversational or workflow agents (Generative AI). Build chatbots/assistants that call APIs and LLMs [1] [8].	Infrastructure management (run commands, patching) on servers/instances [22].	Edge computing (run Lambda functions, local IoT processing).
Runtime Env.	AWS-managed container microVMs (serverless). No on-prem software.	Runs on hosts (EC2 or on-prem); must be installed/updated (often pre-installed on AWS AMIs) [23].	Runs on local devices (embedded Linux). Managed via AWS IoT.
Communication	Invoked via AgentCore API (HTTP/WebSocket) with IAM/OAuth tokens.	Polls AWS SSM service (HTTPS) to receive commands; communicates via `ssmmessages` and `ec2messages` endpoints [22].	Connects via MQTT to AWS IoT Core; messages trigger component actions.
Security	Inbound via Cognito or IAM; each agent session is isolated. VPC mode available [5]. Outbound requests use managed OAuth/API keys.	Uses instance IAM role to authenticate with SSM; traffic goes through AWS endpoints. Must secure instance and limit SSM permissions.	Uses device certificates for authentication. Greengrass core has its own IAM role for AWS resources.
Development	Agent logic in Python/frameworks or any containerizable code. Integrates LangGraph, Strands, etc. [15] [16].	Scripts or packages (PowerShell, Bash, Python, etc.) that run on the OS. Written as SSM Documents.	AWS Lambda functions (Python, Node.js, etc.) running locally on the device.
Tool Integration	Unified Gateway supports any REST/Lambda tool with semantic search [8].	Limited to builtin SSM features (Run Command, State Manager). Cannot natively call arbitrary APIs.	Can call AWS services via local Lambda or MQTT. Less focus on discovery of external enterprise APIs.

Feature/Aspect	**AgentCore (Bedrock)**	**SSM Agent (Systems Manager)**	**Greengrass Core (IoT Edge)**
Logging/ Monitor	CloudWatch Logs/Metrics and X-Ray by default [6]. AgentCore dashboard (GenAI observability).	Logs can be sent to CloudWatch Logs or S3. Basic SSM Dashboard in console.	Logs to CloudWatch (if enabled) or device. Greengrass logs/ debug per component.
Scalability	Auto-scaling serverless model. Easy to deploy many agents/sessions [2].	Dependent on instance capacity. You must manage scaling of underlying servers.	Scales by adding more devices. No auto-scaling; each device is independent.

Conclusion

Amazon Bedrock AgentCore is AWS's full-stack solution for agentic AI. It abstracts the entire infrastructure (compute, state management, tool integration, and security) into a managed service, allowing developers to focus on agent logic [1] [2]. Compared to legacy agents like SSM or Greengrass, AgentCore is entirely serverless and tailored for generative AI workflows. We recommend ensuring least-privilege IAM, using VPC mode if accessing sensitive resources, and monitoring via the built-in CloudWatch dashboards. As AWS continues adding features (policy enforcement, evaluations, episodic memory, etc. are already GA [10] [34]), AgentCore is positioned as the go-to platform for deploying AI agents in production.

Sources: Official AWS documentation (Amazon Bedrock AgentCore DevGuide and FAQs) and AWS announcements [1] [3] [7] [8] [6] [4], as well as AWS blog posts and release notes [2] [10], were used to compile this report.

---

[1] [5] Amazon Bedrock AgentCore- AWS
https://aws.amazon.com/bedrock/agentcore/

[2] [6] [12] Accelerate agentic application development with a full-stack starter template for Amazon Bedrock AgentCore | Artificial Intelligence
https://aws.amazon.com/blogs/machine-learning/accelerate-agentic-application-development-with-a-full-stack-starter-template-for-amazon-bedrock-agentcore/

[3] [9] [15] [19] How it works - Amazon Bedrock AgentCore
https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-how-it-works.html

[4] [24] [26] [30] [32] IAM Permissions for AgentCore Runtime - Amazon Bedrock AgentCore
https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-permissions.html

[7] [20] [21] Add memory to your Amazon Bedrock AgentCore agent - Amazon Bedrock AgentCore
https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/memory.html

[8] [16] Amazon Bedrock AgentCore Gateway: Securely connect tools and other resources to your Gateway - Amazon Bedrock AgentCore
https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/gateway.html

[10] [11] [28] [29] [34] New Amazon Bedrock AgentCore capabilities power the next wave of agentic AI development
https://www.aboutamazon.com/news/aws/aws-amazon-bedrock-agent-core-ai-agents

[13] [14] [25] [31] [33] AgentCore CLI - Amazon Bedrock AgentCore
https://aws.github.io/bedrock-agentcore-starter-toolkit/api-reference/cli.html

[17] Get started with Amazon Bedrock AgentCore - Amazon Bedrock AgentCore
https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/agentcore-get-started-toolkit.html

[18] Supported AWS Regions - Amazon Bedrock AgentCore
https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/agentcore-regions.html

[22] Working with SSM Agent - AWS Systems Manager
https://docs.aws.amazon.com/systems-manager/latest/userguide/ssm-agent.html

[23] Working with SSM Agent on EC2 instances for Windows Server - AWS Systems Manager
https://docs.aws.amazon.com/systems-manager/latest/userguide/ssm-agent-windows.html

[27] AWS Agents Explained: From SSM to Amazon Bedrock AgentCore
https://www.switchsoftware.io/post/the-rise-of-agents-in-aws-and-their-impact-on-cloud-operations